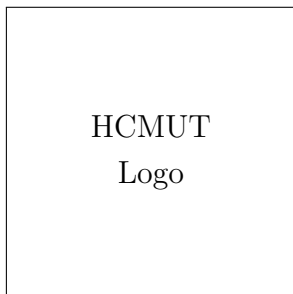


ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO

KIẾN TRÚC PHẦN MỀM

INTELLIGENT RESTAURANT MANAGEMENT SYSTEM (IRMS)

GV: Trần Trương Tuấn Phát

SVTH: Nguyễn Nhật Tiến (2213059)

Lê Bảo Linh (2311849)

Trần Phú (1952924)

Phạm Đình Chương (2210400)

Trần Minh Đăng (2310735)

Hà Đình Vinh (2313907)

TP. HỒ CHÍ MINH, THÁNG 5 NĂM 2026

Mục lục

1	Giới thiệu	3
2	Thiết kế kiến trúc hệ thống	3
2.1	Phân tích yêu cầu	3
2.1.1	Phạm vi dự án	3
2.1.2	Stakeholders	5
2.1.3	Yêu cầu chức năng	5
2.1.4	Yêu cầu phi chức năng	7
2.1.5	Ràng buộc	7
2.2	Đặc trưng kiến trúc và tiêu chí thành công	8
2.2.1	Các đặc trưng kiến trúc quan trọng	8
2.2.2	Tiêu chí đánh giá sự thành công của hệ thống	8
2.3	Chọn kiểu kiến trúc và mẫu kiến trúc	9
2.3.1	So sánh và chọn kiểu kiến trúc	9
2.3.2	Chọn mẫu kiến trúc	11
2.3.3	Kết luận lựa chọn	11
2.4	Thiết kế kiến trúc hệ thống	11
2.4.1	Xác định các actors và actions	11
2.4.2	Xác định các components chính	12
2.4.3	Package Diagram (Module View)	13
2.4.4	Component Diagram (C&C View)	14
2.4.5	Deployment Diagram (Allocation View)	15
2.5	Các quyết định kiến trúc và nguyên tắc thiết kế	16
2.5.1	Các quyết định kiến trúc chính	16
2.5.2	Các nguyên tắc thiết kế được áp dụng	16
2.6	Lược đồ lớp UML	17
2.6.1	Order Module	17
2.6.2	Kitchen Module	18
2.6.3	Billing Module	19
2.6.4	Table Module	20
2.6.5	Admin & Access Module	21
2.7	Áp dụng SOLID Principles vào thiết kế	22
2.7.1	Single Responsibility Principle (SRP)	22
2.7.2	Open/Closed Principle (OCP)	23
2.7.3	Liskov Substitution Principle (LSP)	23
2.7.4	Interface Segregation Principle (ISP)	23
2.7.5	Dependency Inversion Principle (DIP)	24
2.8	Khả năng mở rộng trong tương lai	24

3	Hiện thực hệ thống	24
3.1	Phạm vi hiện thực	24
3.1.1	Các module được hiện thực	24
3.1.2	Luồng nghiệp vụ end-to-end được demo	25
3.2	Công nghệ sử dụng	26
3.3	Cấu trúc thư mục	26
3.4	Tính SOLID trong mã nguồn	27
3.4.1	Single Responsibility Principle (SRP)	27
3.4.2	Open/Closed Principle (OCP)	27
3.4.3	Liskov Substitution Principle (LSP)	27
3.4.4	Interface Segregation Principle (ISP)	28
3.4.5	Dependency Inversion Principle (DIP)	28
3.5	Kết quả hiện thực	28
3.5.1	Màn hình đăng nhập và phân quyền	28
3.5.2	Màn hình quản lý bàn	28
3.5.3	Màn hình lễ tân	29
3.5.4	Màn hình gọi món	29
3.5.5	Màn hình bếp	29
3.5.6	Màn hình lập bill và thanh toán	29
3.5.7	Luồng demo end-to-end	29
3.6	Đóng gói và triển khai	30
4	Báo cáo phản ánh	31
4.1	Lợi ích của việc áp dụng SOLID	31
4.2	Khó khăn gặp phải	31
4.3	Bài học rút ra	31
5	Phân chia công việc	32
6	Tổng kết	32
7	Tài liệu tham khảo	33

1. Giới thiệu

Hệ thống **Intelligent Restaurant Management System (IRMS)** là một giải pháp phần mềm nhằm số hóa và tối ưu hóa các hoạt động cốt lõi trong nhà hàng, bao gồm tiếp nhận order, điều phối bếp, quản lý bàn, lập hóa đơn và thanh toán. Hệ thống đóng vai trò như một nền tảng trung tâm giúp đồng bộ thông tin giữa nhân viên phục vụ, nhân viên bếp, lễ tân và quản lý, từ đó giảm sai sót, rút ngắn thời gian phục vụ và nâng cao trải nghiệm khách hàng.

Trong bối cảnh vận hành nhà hàng hiện đại, việc phối hợp giữa khu vực phục vụ và khu vực bếp thường phát sinh nhiều vấn đề như ghi sai món, chậm cập nhật trạng thái món ăn, thất lạc yêu cầu đặc biệt của khách hàng hoặc chậm trễ trong thanh toán. IRMS được xây dựng để giải quyết các bất cập đó thông qua một hệ thống thống nhất, hỗ trợ xử lý nghiệp vụ theo thời gian thực, đồng thời cung cấp nền tảng mở rộng cho các chức năng quản trị và phân tích vận hành.

Mục tiêu của dự án gồm các nội dung chính sau:

- Tăng hiệu quả vận hành của nhà hàng bằng cách tự động hóa luồng gọi món, chế biến và thanh toán.
- Cải thiện khả năng phối hợp giữa nhân viên phục vụ, nhân viên bếp và lễ tân thông qua dữ liệu đồng bộ theo thời gian thực.
- Thiết kế hệ thống có cấu trúc mô-đun, dễ bảo trì, dễ mở rộng và phù hợp với các nguyên lý SOLID.
- Xây dựng một kiến trúc đủ rõ ràng để mô tả qua nhiều góc nhìn như *module view*, *component-and-connector view* và *allocation view*.
- Hiện thực các chức năng cốt lõi nhằm chứng minh tính khả thi của thiết kế kiến trúc đề xuất.

Với quy mô nhóm gồm 6 thành viên và thời gian thực hiện khoảng 1 tháng, hệ thống được định hướng theo kiến trúc **Layered Modular Monolith kết hợp Event-Driven nội bộ**. Đây là lựa chọn cân bằng giữa tính học thuật trong thiết kế kiến trúc, khả năng hiện thực trong thời gian ngắn và mức độ phù hợp với tiêu chí chấm điểm của học phần.

2. Thiết kế kiến trúc hệ thống

2.1. Phân tích yêu cầu

2.1.1. Phạm vi dự án

Phạm vi của dự án IRMS tập trung vào các luồng nghiệp vụ cốt lõi trong nhà hàng phục vụ tại chỗ (*dine-in restaurant*). Mục tiêu là xây dựng một hệ thống đủ đầy để mô tả kiến

trúc phần mềm, đồng thời thu hẹp phạm vi hiện thực để đảm bảo có thể hoàn thành phần code và demo trong thời gian cho phép.

Các module chính

- **Order Management:** tạo order cho bàn, thêm/xóa món, cập nhật số lượng, thêm ghi chú đặc biệt và theo dõi trạng thái order.
- **Kitchen Workflow:** tiếp nhận order từ khu vực phục vụ, tạo kitchen ticket, cập nhật trạng thái chế biến và phản hồi món đã sẵn sàng.
- **Billing & Payment:** tạo hóa đơn từ order, tính tổng tiền, tip, thanh toán toàn bộ hoặc chia bill theo từng phần.
- **Menu Management:** quản lý danh sách món ăn, giá bán, danh mục món và trạng thái còn bán/hết món.
- **Table Management:** quản lý trạng thái bàn, gán order cho bàn và giải phóng bàn sau khi thanh toán.
- **Reservation & Host:** tiếp nhận đặt bàn, giữ bàn, cho khách đặt trước vào bàn và xử lý khách walk-in chưa đặt trước.
- **User & Access Control:** quản lý tài khoản, vai trò, xác thực và phân quyền.
- **Inventory Monitoring:** theo dõi tồn kho và cảnh báo thiếu hụt.
- **Analytics & Reports:** thống kê doanh thu, đơn hàng và báo cáo vận hành.

Các nâng cấp trong tương lai

- Reservation nâng cao theo khung giờ, nhắc lịch và danh sách chờ
- Employee Scheduling
- Loyalty Program
- Multi-branch support
- AI-based demand forecasting
- Inventory optimization

Các yếu tố loại trừ

- Tích hợp trực tiếp với phần cứng POS chuyên dụng.
- Tích hợp cổng thanh toán thực tế ở mức production.
- Machine learning hoặc demand forecasting nâng cao ở phiên bản đầu tiên.
- Hỗ trợ đa tenant cho chuỗi nhà hàng lớn ngay từ phiên bản đầu tiên.

2.1.2. Stakeholders

Các bên liên quan của hệ thống được chia thành hai nhóm chính: stakeholders trực tiếp sử dụng hệ thống trong vận hành hàng ngày và stakeholders gián tiếp quan tâm đến hiệu quả vận hành, bảo trì và mở rộng hệ thống.

Stakeholders chính

- **Server/Waiter:** tiếp nhận order, theo dõi trạng thái món, hỗ trợ khách hàng và phối hợp với bếp.
- **Chef/Kitchen Staff:** xem danh sách món cần chế biến, cập nhật trạng thái chuẩn bị và hoàn thành món.
- **Manager:** quản lý menu, theo dõi hoạt động vận hành, cập nhật giá và xem báo cáo cơ bản.
- **Host:** điều phối khách, quản lý khu vực bàn, tiếp nhận đặt bàn, giữ bàn và cho khách walk-in vào bàn.
- **Customer:** là đối tượng nhận dịch vụ ăn tại chỗ, gọi món và thanh toán.

Stakeholders phụ

- **System Administrator:** bảo trì hệ thống, cấu hình người dùng và xử lý sự cố kỹ thuật.
- **Restaurant Owner:** quan tâm tới hiệu quả vận hành, doanh thu, khả năng mở rộng lâu dài.
- **IT Support:** hỗ trợ triển khai, sao lưu dữ liệu và giám sát hệ thống.

2.1.3. Yêu cầu chức năng

1. Quản lý bàn

- Hệ thống phải cho phép xem danh sách bàn và trạng thái hiện tại của từng bàn.
- Trạng thái lưu ở backend gồm AVAILABLE, OCCUPIED, RESERVED và CLEANING; giao diện phục vụ suy diễn thêm các trạng thái nghiệp vụ như chờ món, chờ phục vụ hoặc đã phục vụ dựa trên order/kitchen ticket đang mở.
- Lễ tân có thể giữ bàn cho khách đặt trước hoặc cho khách walk-in vào bàn; thay đổi này phải được phản ánh trên màn hình nhân viên phục vụ.
- Khi khách ngồi vào bàn, nhân viên phục vụ có thể tạo order mới gắn với bàn tương ứng.
- Một bàn có thể có nhiều order theo thời gian, nhưng chỉ có một order đang hoạt động tại một thời điểm.

2. Quản lý menu

- Hệ thống phải hiển thị danh sách món ăn theo danh mục.
- Mỗi món ăn cần có tên món, giá bán, mô tả ngắn và trạng thái còn bán/hết món.
- Quản lý có thể thêm mới, chỉnh sửa hoặc ẩn món khỏi menu.
- Khi món ăn được đánh dấu hết hàng, thay đổi đó phải phản ánh ngay trong luồng gọi món.

3. Quản lý order

- Nhân viên phục vụ có thể tạo order cho một bàn cụ thể.
- Một order bao gồm nhiều món, số lượng từng món và các ghi chú đặc biệt như mức cay, dị ứng hoặc yêu cầu riêng của khách.
- Hệ thống phải hỗ trợ chỉnh sửa order trước khi bill được chốt, bao gồm thêm món, xóa món hoặc thay đổi số lượng.
- Order có thể chuyển qua các trạng thái như **Pending**, **Confirmed**, **InPreparation**, **Ready**, **Served**, **Closed**, **Cancelled**.

4. Kitchen Workflow

- Khi order được xác nhận, hệ thống phải tạo *kitchen ticket* để chuyển thông tin sang khu vực bếp.
- Nhân viên bếp có thể xem các món cần chế biến theo thứ tự hoặc theo nhóm danh mục món.
- Hệ thống phải cho phép cập nhật trạng thái từng món như **Queued**, **Preparing**, **Ready**.
- Màn hình bếp cần có danh sách các món đang chờ xử lý và lịch sử các món đã hoàn thành/đã phục vụ để nhân viên kiểm tra lại khi cần.
- Khi toàn bộ món trong order đã hoàn thành, hệ thống cập nhật trạng thái order tương ứng để phục vụ mang món ra bàn.

5. Billing & Payment

- Nhân viên phục vụ có thể tạo bill từ order đang hoạt động và xử lý thanh toán tại bàn.
- Hệ thống phải tính được *subtotal*, phí dịch vụ, giảm giá và tổng tiền thanh toán.
- Hệ thống hỗ trợ ít nhất các phương thức thanh toán: tiền mặt, thẻ và ví điện tử.

-
- Hệ thống hỗ trợ thanh toán toàn bộ phần còn lại hoặc split bill theo số tiền từng lần thanh toán.
 - Sau khi thanh toán thành công, order được đóng và bàn được chuyển về trạng thái phù hợp để tiếp tục phục vụ.

6. Quản trị và phân quyền

- Hệ thống phải hỗ trợ các vai trò đang dùng trong demo: **Admin**, **Manager**, **Server**, **Chef**, **Host**. Chức năng thu tiền được gộp vào vai trò **Server** trong MVP.
- Mỗi vai trò chỉ được truy cập các chức năng phù hợp với trách nhiệm nghiệp vụ.
- Hệ thống cần ghi nhận các hành động quan trọng như hủy order, cập nhật giá món hoặc hoàn tất thanh toán để hỗ trợ kiểm tra về sau.

2.1.4. Yêu cầu phi chức năng

- **Hiệu năng**: thời gian phản hồi cho các thao tác phổ biến như xem menu, tạo order, xem bill nên nhỏ hơn 2 giây trong điều kiện bình thường.
- **Tính nhất quán**: trạng thái order, trạng thái bill và trạng thái bàn phải luôn đồng bộ sau mỗi thao tác nghiệp vụ.
- **Tính sẵn sàng**: hệ thống cần hoạt động ổn định trong giờ cao điểm của nhà hàng.
- **Khả năng sử dụng**: giao diện phải đủ trực quan để nhân viên mới có thể làm quen nhanh.
- **Khả năng bảo trì**: mã nguồn cần có cấu trúc rõ ràng, dễ test, dễ thay đổi và tuân thủ các nguyên lý SOLID.
- **Khả năng mở rộng**: dễ dàng bổ sung các module như Employee Scheduling, Loyalty Program hoặc reservation nâng cao trong tương lai.
- **Bảo mật**: người dùng phải đăng nhập trước khi sử dụng hệ thống và chỉ truy cập được các chức năng được phân quyền.
- **Khả năng kiểm toán**: các thao tác nghiệp vụ quan trọng cần được ghi nhận để hỗ trợ tra cứu khi cần.

2.1.5. Ràng buộc

- Thời gian thực hiện dự án khoảng 1 tháng.
- Nhóm thực hiện gồm 6 thành viên.

-
- Hạ tầng triển khai và thời gian cấu hình có giới hạn, nên không phù hợp với kiến trúc phân tán phức tạp.
 - Demo cần hoạt động ổn định, dễ trình bày và bám sát luồng nghiệp vụ thực tế của nhà hàng.
 - Phần hiện thực không bắt buộc bao phủ toàn bộ phạm vi lý thuyết, mà ưu tiên một luồng end-to-end hoàn chỉnh.

2.2. Đặc trưng kiến trúc và tiêu chí thành công

2.2.1. Các đặc trưng kiến trúc quan trọng

Từ yêu cầu nghiệp vụ và các ràng buộc thực tế, nhóm xác định các đặc trưng kiến trúc cốt lõi của IRMS như sau:

- **Maintainability:** hệ thống phải dễ sửa đổi khi thay đổi nghiệp vụ, ví dụ điều chỉnh luồng order, thêm rule giảm giá hoặc thay đổi cách xử lý trạng thái món.
- **Extensibility:** kiến trúc cần cho phép thêm các module như Loyalty, Employee Scheduling hoặc reservation nâng cao mà không phá vỡ cấu trúc lõi.
- **Consistency:** trạng thái của order, bill và table phải nhất quán để tránh sai lệch dữ liệu vận hành.
- **Performance:** các thao tác trực tiếp phục vụ nhân viên như xem menu, tạo order, cập nhật trạng thái món phải phản hồi nhanh.
- **Usability:** giao diện và workflow cần đơn giản để phù hợp với môi trường phục vụ tốc độ cao.
- **Security:** chỉ người dùng được phân quyền mới có thể truy cập các chức năng tương ứng.
- **Testability:** hệ thống cần được thiết kế sao cho có thể kiểm thử theo từng module và từng use case quan trọng.

2.2.2. Tiêu chí đánh giá sự thành công của hệ thống

IRMS được xem là thành công nếu đáp ứng được các tiêu chí sau:

- Nhân viên có thể hoàn thành luồng chính: chọn bàn → tạo order → bếp cập nhật trạng thái → tạo bill → thanh toán.
- Trạng thái order, bàn và bill không bị lệch trong suốt luồng nghiệp vụ.
- Việc thêm phương thức thanh toán hoặc chính sách giảm giá mới không yêu cầu sửa logic lõi quá nhiều.

-
- Các module chính có ranh giới rõ ràng, dễ phân công công việc và dễ bảo trì.
 - Hệ thống có thể mở rộng thêm ít nhất một module mới trong tương lai mà không cần thiết kế lại toàn bộ kiến trúc.

2.3. Chọn kiểu kiến trúc và mẫu kiến trúc

Dựa trên yêu cầu chức năng, yêu cầu phi chức năng và các ràng buộc thực tế của nhóm, có thể xác định một số đặc trưng kiến trúc quan trọng đối với IRMS như sau:

- **Maintainability:** hệ thống phải dễ sửa đổi và dễ bảo trì vì nhiều nghiệp vụ có thể thay đổi trong quá trình phát triển.
- **Extensibility:** hệ thống cần hỗ trợ mở rộng các module như Employee Scheduling, Loyalty Program hoặc reservation nâng cao trong tương lai.
- **Consistency:** dữ liệu giữa order, kitchen, billing và table phải đồng bộ.
- **Usability:** giao diện và luồng xử lý cần đơn giản cho người dùng cuối.
- **Performance:** các thao tác nghiệp vụ chính cần có phản hồi nhanh.
- **Testability:** dễ viết unit test và integration test cho các module chính.

2.3.1. So sánh và chọn kiểu kiến trúc

1. Monolithic truyền thống

Ưu điểm

- Đơn giản để khởi động.
- Dễ triển khai trong một môi trường duy nhất.
- Hiệu năng nội bộ tốt do không có giao tiếp mạng giữa các dịch vụ.

Nhược điểm

- Nếu không chia ranh giới rõ ràng, hệ thống dễ trở thành một khối lớn khó bảo trì.
- Khó phân chia công việc song song cho nhiều thành viên.
- Khó chứng minh rõ tính mô-đun trong báo cáo kiến trúc.

2. Microservices

Ưu điểm

- Tách biệt dịch vụ độc lập, dễ mở rộng theo từng phần.
- Dễ triển khai riêng các thành phần khi hệ thống phát triển lớn.
- Tạo ranh giới dịch vụ rõ ràng nếu được đầu tư đầy đủ về hạ tầng.

Nhược điểm

- Tăng đáng kể độ phức tạp triển khai, cấu hình và tích hợp.
- Đòi hỏi nhiều công sức cho tracing, service discovery, API gateway, xử lý lỗi mạng và đồng bộ dữ liệu.
- Không phù hợp với nhóm nhỏ và thời gian ngắn nếu mục tiêu là hoàn thành một demo ổn định.

3. Layered Modular Monolith

Ưu điểm

- Toàn hệ thống chạy như một ứng dụng duy nhất nên dễ build, test và demo.
- Có thể chia thành các module rõ ràng để phân công công việc.
- Phù hợp với việc áp dụng SOLID và mô tả bằng *module view*.
- Có khả năng chuyển đổi dần sang microservices trong tương lai nếu cần.

Nhược điểm

- Cần kỷ luật thiết kế tốt để tránh phụ thuộc chéo không kiểm soát giữa các module.
- Không hỗ trợ scale độc lập từng module tốt như microservices.

Từ các phân tích trên, nhóm lựa chọn **Layered Modular Monolith** là kiến trúc chính của IRMS. Đây là phương án phù hợp nhất với quy mô nhóm, thời gian thực hiện và mục tiêu cân bằng giữa chất lượng thiết kế kiến trúc và khả năng hiện thực.

2.3.2. Chọn mẫu kiến trúc

Nhóm chọn kết hợp ba hướng tổ chức sau:

- **Layered Architecture** làm khung phân lớp chính.
- **Modular Monolith** làm hình thái triển khai tổng thể.
- **Internal Event-Driven** làm cơ chế phối hợp nội bộ giữa các module.

Hệ thống được tổ chức thành bốn lớp:

- **Presentation Layer**: tiếp nhận thao tác từ giao diện người dùng.
- **Application Layer**: điều phối các use case.
- **Domain Layer**: chứa entity, business rules, interface và các domain service.
- **Infrastructure Layer**: hiện thực truy cập dữ liệu, cơ chế lưu trữ, event dispatcher và các adapter kỹ thuật.

Ngoài ra, các sự kiện nội bộ như `OrderPlaced`, `OrderUpdated`, `KitchenItemReady`, `PaymentCompleted` được sử dụng để giảm độ phụ thuộc trực tiếp giữa các module nhưng vẫn không làm tăng độ phức tạp như message broker phân tán.

2.3.3. Kết luận lựa chọn

Nhóm quyết định sử dụng kiến trúc **Layered Modular Monolith kết hợp Event-Driven nội bộ** cho IRMS vì các lý do sau:

- Phù hợp với thời gian thực hiện và quy mô nhóm.
- Dễ hiện thực và dễ demo hơn so với microservices.
- Vẫn đủ mạnh để trình bày học thuật qua các góc nhìn kiến trúc khác nhau.
- Hỗ trợ tốt việc áp dụng SOLID trong cả thiết kế lẫn mã nguồn.
- Dễ mở rộng để bổ sung module mới trong tương lai.

2.4. Thiết kế kiến trúc hệ thống

2.4.1. Xác định các actors và actions

Dựa trên yêu cầu nghiệp vụ, hệ thống có các actor và các hành động chính như sau:

Bảng 1: Các actor và actions chính trong IRMS

Actor	Actions
Server/Waiter	createOrder, updateOrder, addItem, removeItem, viewOrderStatus, assignTable
Chef/Kitchen Staff	viewKitchenTickets, startPreparation, markItemReady, updateKitchenStatus
Manager	manageMenu, updatePrice, viewOperations, viewBasicReports, manageUsers
Host	assignTable, manageReservations, updateReservationStatus, seatWalkInGuest
System	publishOrderPlaced, generateKitchenTicket, updateTableStatus

2.4.2. Xác định các components chính

Từ các actor và actions đã xác định, hệ thống được chia thành các thành phần chính như sau:

- **Admin & Access Module**
- **Order Module**
- **Kitchen Module**
- **Billing Module**
- **Table Module**
- **Reservation Module**
- **Inventory Module**
- **Analytics Module**
- **Audit Module**
- **Notification Module**
- **Common & Configuration Module**

Trong phiên bản hiện tại, các module Order, Kitchen, Billing, Table, Menu, Reservation/Host và Access Control tạo thành luồng demo chính; Inventory, Analytics và Audit Log hỗ trợ vận hành ở mức backend cơ bản.

Mỗi module có ranh giới trách nhiệm tương đối độc lập, đồng thời giao tiếp với nhau thông qua abstraction hoặc domain event.

2.4.3. Package Diagram (Module View)

Ở mức *module view*, IRMS được tổ chức thành các package chính:

- **admin**
 - authentication, user management, menu management
 - services: `AuthenticationService`, `UserDetailsServiceImpl`
 - controllers: `AuthController`, `UserController`, `MenuItemController`
- **order**
 - entities: `Order`, `OrderItem`, `OrderStatus`
 - services: `IOrderService`, `OrderServiceImpl`, `OrderCalculator`, `OrderValidator`
 - repositories: `OrderRepository`
- **kitchen**
 - entities: `KitchenOrder`, `KitchenOrderStatus`
 - services: `IKitchenService`, `KitchenServiceImpl`, `KitchenOrderTimeoutService`, `KitchenOrderFactory`
 - event listener: `KitchenOrderPlacedEventListener`
 - repositories: `KitchenOrderRepository`
- **billing**
 - entities: `Bill`, `Payment`, `BillStatus`, `PaymentStatus`, `PaymentMethod`
 - services: `IBillingService`, `BillingServiceImpl`, `BillCalculator`, `PaymentProcessor`
 - strategies: `CashPaymentProcessor`, `CardPaymentProcessor`, `EWalletPaymentProcessor`, `DiscountStrategy`
 - repositories: `BillRepository`
- **table**
 - entities: `Table`, `TableStatus`
 - services: `ITableService`, `TableServiceImpl`, `DemoResetService`
 - controller: `TableController`
 - repositories: `TableRepository`
- **reservation**
 - entities và repositories cho đặt bàn

-
- API phục vụ host tạo, cập nhật, giữ bàn, cho khách đặt trước vào bàn và xử lý khách walk-in
 - **notification**
 - websocket notification, event handling, operational alerts
 - **inventory**
 - entities: `InventoryItem`, `InventoryStatus`
 - controller: `InventoryItemController`
 - **analytics**
 - services: `AnalyticsService`
 - controller: `AnalyticsController`
 - **common, config**
 - common interfaces, domain events, exceptions, utilities
 - security, websocket, application configuration

Quan hệ phụ thuộc chính

- Order sử dụng dữ liệu menu từ admin module và Table ở mức đọc dữ liệu.
- Kitchen nhận thông tin từ Order thông qua event hoặc abstraction.
- Billing sử dụng dữ liệu từ Order để sinh Bill.
- Table cập nhật trạng thái dựa trên kết quả của Order và Payment.
- Notification phản ứng với một số sự kiện vận hành như order được tạo hoặc payment hoàn tất.
- Common/Config cung cấp hạ tầng dùng chung cho tất cả các module.

2.4.4. Component Diagram (C&C View)

Ở mức *component-and-connector view*, hệ thống gồm các thành phần chạy chính sau:

- **Admin UI**
- **Server UI**
- **Kitchen Display UI**
- **Host UI**

-
- **Manager UI**
 - **Backend Application**
 - Admin/Auth Component
 - Order Component
 - Kitchen Component
 - Billing Component
 - Table Component
 - Inventory Component
 - Analytics Component
 - Notification Component
 - Event Dispatcher
 - **Relational Database**

Luồng xử lý chính

1. Server UI gửi yêu cầu tạo order đến Backend Application.
2. Order Component lưu order và phát sự kiện `OrderPlaced`.
3. Kitchen Component nhận sự kiện và tạo kitchen ticket.
4. Kitchen Display UI cập nhật trạng thái món trong quá trình chế biến.
5. Billing Component tạo bill từ order.
6. Server UI thực hiện thanh toán tại bàn.
7. Khi thanh toán thành công, hệ thống phát `PaymentCompleted`.
8. Table Component cập nhật trạng thái bàn tương ứng.

2.4.5. Deployment Diagram (Allocation View)

Ở mức *allocation view*, hệ thống được triển khai theo cấu trúc đơn giản như sau:

- **Client Node 1:** Tablet hoặc trình duyệt cho nhân viên phục vụ.
- **Client Node 2:** Màn hình hiển thị bếp.
- **Client Node 3:** Terminal cho lễ tân hoặc quản lý.
- **Client Node 4:** Dashboard cho quản lý.

-
- **Application Server:** chạy ứng dụng backend theo kiến trúc modular monolith.
 - **Database Server:** PostgreSQL, quản lý schema và seed data bằng Flyway migration.

Các client giao tiếp với backend qua HTTP/HTTPS. Backend giao tiếp với database qua kết nối nội bộ. Vì toàn bộ nghiệp vụ chạy trong một ứng dụng backend thống nhất, việc triển khai, theo dõi lỗi và debug trở nên đơn giản hơn đáng kể so với kiến trúc phân tán.

2.5. Các quyết định kiến trúc và nguyên tắc thiết kế

2.5.1. Các quyết định kiến trúc chính

Nhóm đưa ra các quyết định kiến trúc sau:

- Chọn **Layered Modular Monolith** thay vì Microservices để giảm rủi ro tích hợp và tăng xác suất hoàn thành demo trong thời gian ngắn.
- Tổ chức hệ thống theo các **module nghiệp vụ rõ ràng** như Admin, Order, Kitchen, Billing và Table để tăng tính cohesive và hỗ trợ chia việc cho nhóm.
- Sử dụng **domain event nội bộ** để kết nối các module thay vì gọi chéo trực tiếp quá nhiều, ví dụ `OrderPlaced` và `PaymentCompleted`.
- Chọn **relational database** làm nguồn dữ liệu chính nhằm đảm bảo tính nhất quán cho order, bill và trạng thái bàn.
- Tách riêng **frontend** và **backend** để giảm coupling giữa phần giao diện và phần nghiệp vụ.
- Ưu tiên hiện thực một **vertical slice end-to-end** thay vì dàn trải toàn bộ hệ thống.

2.5.2. Các nguyên tắc thiết kế được áp dụng

Các nguyên tắc thiết kế được áp dụng trong IRMS gồm:

- **Separation of Concerns:** mỗi layer và mỗi module chỉ xử lý một nhóm trách nhiệm riêng.
- **High Cohesion, Low Coupling:** các class trong cùng module liên quan chặt chẽ, trong khi coupling giữa các module được giảm tối đa.
- **Interface-first Design:** service và repository phụ thuộc vào abstraction để hỗ trợ test và mở rộng.

-
- **Dependency Injection:** giảm phụ thuộc cứng giữa các class.
 - **Open for Extension, Closed for Modification:** đặc biệt ở các điểm biến đổi như phương thức thanh toán và chính sách giảm giá.
 - **Domain-centered Modeling:** class và module được tổ chức xoay quanh thực thể nghiệp vụ của nhà hàng.

2.6. Lược đồ lớp UML

Do hệ thống được xây dựng theo mô hình modular monolith, việc mô tả class diagram được chia theo từng module chính thay vì chia theo từng service độc lập như mô hình microservices.

2.6.1. Order Module

Class Diagram

Các lớp chính của Order Module bao gồm:

- Order
- OrderItem
- OrderStatus
- OrderServiceImpl
- IOrderService
- OrderCalculator
- OrderValidator
- OrderStatusTransitionValidator
- OrderRepository
- OrderController

Quan hệ chính

- Order bao gồm nhiều OrderItem.
- OrderItem tham chiếu đến MenuItem.
- OrderController phụ thuộc vào abstraction IOrderService.
- OrderServiceImpl phụ thuộc vào OrderRepository, MenuItemRepository, TableRepository, UserRepository và DomainEventPublisher.
- Sau khi tạo order, module phát OrderPlacedEvent để Kitchen Module tạo kitchen ticket.

Nguyên tắc SOLID trong lược đồ

- **SRP**: `OrderController` chỉ xử lý HTTP request/response; `OrderServiceImpl` điều phối use case; `OrderCalculator`, `OrderValidator` và `OrderNumberGenerator` tách riêng các logic nghiệp vụ nhỏ.
- **OCP**: có thể mở rộng logic tạo order bằng các policy mới mà không cần sửa controller.
- **LSP**: các implementation khác nhau của `OrderRepository` có thể thay thế nhau mà không làm hỏng logic nghiệp vụ.
- **ISP**: có thể tách `OrderCommandService` và `OrderQueryService` khi module phát triển lớn hơn.
- **DIP**: service phụ thuộc vào abstraction như repository và event publisher thay vì implementation cụ thể.

Khả năng mở rộng

Order Module có thể được mở rộng để hỗ trợ:

- gộp hoặc tách order cho nhóm khách,
- xử lý combo hoặc set menu,
- hỗ trợ order theo thiết bị riêng của khách hàng,
- ưu tiên order trong giờ cao điểm.

2.6.2. Kitchen Module

Class Diagram

Các lớp chính bao gồm:

- `KitchenOrder`
- `KitchenOrderStatus`
- `KitchenDisplayOrderResponse`
- `KitchenServiceImpl`
- `IKitchenService`
- `KitchenOrderRepository`
- `KitchenOrderPlacedEventListener`
- `KitchenController`

Kitchen Module chịu trách nhiệm tiếp nhận dữ liệu từ Order Module, tổ chức ticket theo góc nhìn của bếp và cập nhật trạng thái chế biến món.

Nguyên tắc SOLID trong lược đồ

- **SRP**: `KitchenServiceImpl` xử lý workflow bếp; `KitchenOrderPlacedEventListener` chỉ nhận sự kiện order mới; `KitchenController` chỉ cung cấp API.
- **OCP**: có thể thêm logic ưu tiên ticket hoặc phân luồng theo trạm bếp mà không làm thay đổi cấu trúc lõi.
- **LSP**: các implementation khác nhau của `KitchenRepository` có thể thay thế lẫn nhau.
- **ISP**: các interface cập nhật trạng thái và truy vấn ticket có thể được tách riêng khi cần.
- **DIP**: service phụ thuộc abstraction của repository và event handler.

Khả năng mở rộng

Kitchen Module có thể mở rộng theo các hướng:

- thêm cơ chế ưu tiên món theo deadline,
- phân chia ticket theo khu vực bếp,
- tích hợp cảnh báo món chậm,
- thống kê thời gian chế biến trung bình.

2.6.3. Billing Module

Class Diagram

Các lớp chính bao gồm:

- `Bill`
- `Payment`
- `PaymentMethod` (enum)
- `PaymentProcessor` (interface)
- `CashPaymentProcessor`, `CardPaymentProcessor`, `EWalletPaymentProcessor`
- `DiscountStrategy` (interface)
- `NoDiscountStrategy`, `PercentageDiscountStrategy`, `FixedAmountDiscountStrategy`, `CouponDiscountStrategy`, `MembershipDiscountStrategy`
- `BillCalculator`, `BillNumberGenerator`, `PaymentStatusCalculator`

-
- `BillingServiceImpl`
 - `IBillingService`
 - `BillRepository`

Billing Module là nơi thể hiện rõ nhiều nguyên lý SOLID, đặc biệt là Strategy và Dependency Inversion.

Nguyên tắc SOLID trong lược đồ

- **SRP**: `BillingServiceImpl` điều phối use case; `BillCalculator` chỉ tính tiền; `PaymentProcessor` chỉ xử lý thanh toán; `BillRepository` chỉ thao tác dữ liệu.
- **OCP**: thêm phương thức thanh toán hoặc chính sách giảm giá mới mà không sửa logic service cũ.
- **LSP**: `CashPaymentProcessor`, `CardPaymentProcessor`, `EWalletPaymentProcessor` thay thế được cho `PaymentProcessor`.
- **ISP**: có thể tách `BillCalculator`, `PaymentProcessor`, `BillingQueryService`.
- **DIP**: service phụ thuộc vào abstraction như `IBillingService`, `PaymentProcessor`, `DiscountStrategy` và `BillRepository`.

Khả năng mở rộng

Billing Module có thể mở rộng để hỗ trợ:

- split bill theo từng món hoặc từng khách,
- rule tính tip nâng cao,
- loyalty discount,
- voucher hoặc campaign theo thời gian,
- tích hợp payment gateway thật.

2.6.4. Table Module

Class Diagram

Các lớp chính bao gồm:

- `Table`
- `TableStatus`
- `TableController`
- `TableRequest`

-
- `TableRepository`

Module này chịu trách nhiệm quản lý bàn và liên kết chặt chẽ với Order cũng như Billing.

Nguyên tắc SOLID trong lược đồ

- **SRP**: `TableService` chỉ xử lý logic bàn; repository chỉ thao tác dữ liệu bàn.
- **OCP**: có thể thêm rule phân bổ bàn hoặc reservation mà không thay đổi toàn bộ module.
- **LSP**: implementation khác nhau của repository vẫn đảm bảo hành vi đúng.
- **ISP**: service truy vấn trạng thái bàn và service cập nhật trạng thái bàn có thể được tách riêng khi cần.
- **DIP**: `TableService` phụ thuộc vào interface repository và event abstraction.

Khả năng mở rộng

Table Module có thể được mở rộng để:

- hỗ trợ reservation theo khung giờ,
- theo dõi sơ đồ bàn theo khu vực,
- tối ưu ghép bàn cho nhóm khách lớn,
- hỗ trợ waitlist.

2.6.5. Admin & Access Module

Class Diagram

Các lớp chính bao gồm:

- `User`
- `Role`
- `Permission`
- `AuthService`
- `AuthenticationService`
- `UserService`

Module này cung cấp khả năng xác thực, quản lý tài khoản và kiểm soát quyền truy cập theo vai trò.

Nguyên tắc SOLID trong lược đồ

- **SRP**: xác thực, quản lý người dùng và phân quyền được tách thành các lớp chịu trách nhiệm riêng.
- **OCP**: có thể thêm vai trò mới mà không cần thay đổi toàn bộ cấu trúc người dùng.
- **LSP**: các strategy xác thực khác nhau có thể thay thế nhau nếu hệ thống mở rộng thêm OAuth hoặc 2FA.
- **ISP**: interface người dùng và xác thực nên được giữ nhỏ gọn theo use case.
- **DIP**: controller phụ thuộc vào service abstraction thay vì lớp cụ thể.

Khả năng mở rộng

Admin & Access Module có thể được mở rộng để:

- hỗ trợ đăng nhập nhiều cơ chế,
- thêm 2FA,
- quản lý permission chi tiết hơn theo hành động,
- audit hoạt động người dùng.

2.7. Áp dụng SOLID Principles vào thiết kế

2.7.1. Single Responsibility Principle (SRP)

Nguyên lý SRP được áp dụng bằng cách tách biệt rõ ràng trách nhiệm của từng lớp và từng module:

- `OrderController` chỉ xử lý request/response liên quan đến order.
- `OrderServiceImpl` điều phối nghiệp vụ tạo và cập nhật order.
- `BillCalculator` chỉ chịu trách nhiệm tính subtotal, tax, service charge và tổng tiền.
- `PaymentProcessor` chỉ xử lý nghiệp vụ thanh toán theo từng phương thức cụ thể.
- `KitchenOrder` đại diện dữ liệu ticket bếp ở mức từng món.
- `DomainEventPublisher` chỉ chịu trách nhiệm phát domain event.

Nhờ vậy, mỗi lớp chỉ có một lý do duy nhất để thay đổi, từ đó giúp hệ thống dễ bảo trì hơn.

2.7.2. Open/Closed Principle (OCP)

Nguyên lý OCP được áp dụng thông qua việc sử dụng abstraction để mở rộng hệ thống mà không phải sửa đổi logic cũ:

- `PaymentProcessor` cho phép bổ sung phương thức thanh toán mới mà không cần sửa luồng xử lý chính của `BillingServiceImpl`.
- `DiscountStrategy` cho phép thêm chính sách giảm giá mới mà không làm thay đổi logic tính bill hiện có.
- Các repository interface cho phép thay đổi cách lưu trữ dữ liệu mà không ảnh hưởng tầng nghiệp vụ.

2.7.3. Liskov Substitution Principle (LSP)

Nguyên lý LSP được áp dụng khi các lớp con hoặc các implementation có thể thay thế abstraction tương ứng mà không làm thay đổi hành vi kỳ vọng của hệ thống:

- `CashPaymentProcessor`, `CardPaymentProcessor`, `EWalletPaymentProcessor` có thể thay thế `PaymentProcessor`.
- `NoDiscountStrategy`, `PercentageDiscountStrategy`, `FixedAmountDiscountStrategy`, `CouponDiscountStrategy` và `MembershipDiscountStrategy` có thể thay thế `DiscountStrategy`.
- Các implementation repository đều phải tuân thủ hợp đồng của interface tương ứng.

2.7.4. Interface Segregation Principle (ISP)

Thay vì xây dựng một interface lớn cho mọi use case, hệ thống được chia thành nhiều interface nhỏ và chuyên biệt:

- `IOrderService`
- `IKitchenService`
- `IBillingService`
- `PaymentProcessor`
- `BillCalculator`

Cách tiếp cận này giúp mỗi lớp chỉ phụ thuộc vào các phương thức mà nó thực sự cần dùng.

2.7.5. Dependency Inversion Principle (DIP)

Nguyên lý DIP được áp dụng bằng cách để các lớp ở mức cao phụ thuộc vào abstraction thay vì implementation cụ thể:

- Service phụ thuộc vào `OrderRepository`, `BillRepository`, `KitchenOrderRepository`, `TableRepository` và `DomainEventPublisher`.
- `BillingServiceImpl` lấy `PaymentProcessor` thông qua `PaymentProcessorFactory`.
- Controller phụ thuộc vào interface service thay vì lớp implementation cụ thể.

Nhờ đó, hệ thống dễ kiểm thử hơn, dễ thay thế implementation hơn và phù hợp với cơ chế Dependency Injection.

2.8. Khả năng mở rộng trong tương lai

Kiến trúc hiện tại cho phép mở rộng theo nhiều hướng mà không phá vỡ cấu trúc lõi:

- Mở rộng **Reservation Module** với nhắc lịch, danh sách chờ và tối ưu bàn theo khung giờ.
- Thêm **Employee Scheduling Module** để quản lý ca làm việc và phân công nhân sự.
- Thêm **Loyalty Module** để tích điểm và chăm sóc khách hàng thân thiết.
- Tích hợp payment gateway thực tế.
- Bổ sung notification đa kênh như email, SMS hoặc push notification.
- Tách dần một số module thành microservice nếu quy mô hệ thống tăng mạnh trong tương lai.

Ví dụ, phần reservation nâng cao có thể kết nối với **Table Module** để tối ưu phân bổ bàn theo khung giờ, trong khi **Loyalty Module** có thể lấy dữ liệu từ **Bill** và **Order** để tính điểm thưởng mà không làm ảnh hưởng trực tiếp tới các module lõi hiện tại.

3. Hiện thực hệ thống

3.1. Phạm vi hiện thực

3.1.1. Các module được hiện thực

Để cân bằng giữa mục tiêu học thuật và khả năng hoàn thành trong thời gian ngắn, nhóm đã hiện thực đầy đủ các module lõi sau:

- **Order Module**

-
- **Kitchen Module**
 - **Billing Module**
 - **Table Module**
 - **Admin & Access Module**
 - **Menu Management**
 - **Reservation/Host Module**

Ngoài ra, hệ thống có hiện thực backend cơ bản cho:

- **Inventory Module**
- **Analytics Module**
- **Audit Log Module**

Các module Employee Scheduling, Loyalty, multi-branch và reservation nâng cao vẫn ở mức thiết kế hoặc kế hoạch mở rộng, chưa nằm trong phạm vi MVP.

3.1.2. Luồng nghiệp vụ end-to-end được demo

Nhóm tập trung hiện thực một vertical slice xuyên suốt toàn bộ hệ thống như sau:

1. Lễ tân giữ bàn cho khách đặt trước hoặc cho khách walk-in vào bàn.
2. Nhân viên phục vụ chọn bàn đang có khách.
3. Tạo order và thêm món.
4. Hệ thống sinh kitchen ticket cho bếp.
5. Nhân viên bếp cập nhật trạng thái món.
6. Nhân viên phục vụ có thể tiếp tục thêm món lần hai trước khi thanh toán.
7. Nhân viên phục vụ tạo bill.
8. Xử lý thanh toán toàn bộ hoặc split bill.
9. Hệ thống đóng order và giải phóng bàn.

Luồng này phản ánh rõ giá trị cốt lõi của IRMS và là cơ sở chính cho demo.

3.2. Công nghệ sử dụng

Stack công nghệ được sử dụng trong phiên bản hiện thực gồm:

- **Backend:** Java 17, Spring Boot, Spring Security, Spring Data JPA, WebSocket.
- **Frontend:** ReactJS, JavaScript, Vite, Axios.
- **Database:** PostgreSQL.
- **ORM:** Spring Data JPA / Hibernate.
- **Migration:** Flyway.
- **Authentication:** JWT
- **Containerization:** Docker Compose cho backend và database.

Việc sử dụng Spring Boot đặc biệt phù hợp với kiến trúc modular monolith vì framework này hỗ trợ tốt Dependency Injection, layered architecture, interface-based design và kiểm thử theo module.

3.3. Cấu trúc thư mục

Ví dụ cấu trúc thư mục chính của IRMS như sau:

```
|-- backend/
|   |-- src/main/java/com/irms/
|   |   |-- admin/
|   |   |-- order/
|   |   |-- kitchen/
|   |   |-- billing/
|   |   |-- table/
|   |   |-- reservation/
|   |   |-- inventory/
|   |   |-- analytics/
|   |   |-- audit/
|   |   |-- notification/
|   |   |-- common/
|   |   '-- config/
|   |-- src/main/resources/
|   |   '-- db/migration/
|   '-- pom.xml
|-- frontend/
|   |-- src/
```

```
| | |-- components/  
| | |-- services/  
| | |-- hooks/  
| | '-- pages/  
| '-- package.json  
'-- report.tex
```

Cấu trúc này phản ánh cách chia theo module nghiệp vụ thay vì chia thành nhiều service độc lập. Nhờ đó, hệ thống vẫn giữ được tính mô-đun nhưng dễ hiện thực và dễ triển khai hơn.

3.4. Tính SOLID trong mã nguồn

3.4.1. Single Responsibility Principle (SRP)

Trong mã nguồn, SRP được thể hiện thông qua việc:

- Controller chỉ nhận request và trả response.
- Service chỉ xử lý logic nghiệp vụ.
- Repository chỉ thực hiện truy cập dữ liệu.
- Các lớp như `KitchenOrderFactory`, `TableServiceImpl` và `DemoResetService` tách riêng trách nhiệm tạo ticket bếp, điều phối nghiệp vụ bàn và reset dữ liệu demo.
- Mapper và Validator có thể được tách riêng nếu hệ thống phát triển lớn hơn.

3.4.2. Open/Closed Principle (OCP)

OCP trong mã nguồn được thể hiện rõ nhất ở:

- Việc bổ sung lớp `MomoPaymentProcessor` hoặc `BankTransferPaymentProcessor` thông qua interface `PaymentProcessor`.
- Việc bổ sung chiến lược giảm giá mới thông qua interface `DiscountStrategy`.
- Việc bổ sung cách tạo kitchen ticket mới có thể đặt vào factory hoặc service chuyên trách thay vì thay đổi controller/order service.

3.4.3. Liskov Substitution Principle (LSP)

LSP trong mã nguồn yêu cầu mọi lớp con hoặc implementation mới phải giữ nguyên hợp đồng hành vi của abstraction. Chẳng hạn, mọi lớp triển khai `PaymentProcessor` đều phải xử lý thanh toán theo giao diện thống nhất và có thể được dùng thay thế lẫn nhau trong cùng một ngữ cảnh.

3.4.4. Interface Segregation Principle (ISP)

ISP được thể hiện qua việc tránh sử dụng interface quá lớn. Các interface được tách theo use case:

- `IOrderService` cho các use case order.
- `IKitchenService` cho workflow bếp.
- `IBillingService` cho nghiệp vụ bill và payment.
- `ITableService` cho nghiệp vụ bàn thay vì để controller thao tác trực tiếp repository.
- Interface riêng cho xử lý payment và discount strategy.

3.4.5. Dependency Inversion Principle (DIP)

DIP được áp dụng bằng cách:

- Service inject repository interface.
- Controller inject service interface.
- `OrderServiceImpl` chỉ phát `OrderPlacedEvent`; `KitchenOrderPlacedEventListener` thuộc Kitchen Module lắng nghe sự kiện để giảm phụ thuộc trực tiếp từ Order sang Kitchen.
- Dễ dàng mock abstraction trong unit test hoặc integration test.

3.5. Kết quả hiện thực

3.5.1. Màn hình đăng nhập và phân quyền

Hệ thống cung cấp màn hình đăng nhập cho người dùng theo từng vai trò. Sau khi xác thực thành công, người dùng được điều hướng đến đúng giao diện tương ứng với vai trò của họ. Điều này giúp bảo vệ chức năng khỏi truy cập trái phép và phản ánh đúng yêu cầu RBAC trong thiết kế.

3.5.2. Màn hình quản lý bàn

Màn hình quản lý bàn hiển thị trạng thái hiện tại của từng bàn, cho phép nhân viên phục vụ hoặc host chọn bàn phù hợp trước khi tạo order. Backend lưu các trạng thái bàn ổn định như `AVAILABLE`, `OCCUPIED`, `RESERVED` và `CLEANING`; frontend suy diễn thêm trạng thái nghiệp vụ như có khách chưa gọi món, chờ món, chờ phục vụ hoặc đã phục vụ để hỗ trợ quyết định nhanh trong môi trường phục vụ thực tế.

3.5.3. Màn hình lễ tân

Màn hình lễ tân hỗ trợ hai luồng đón khách: khách đã đặt trước và khách walk-in. Với khách đặt trước, lễ tân có thể tạo reservation, xác nhận, gán bàn để chuyển bàn sang trạng thái **RESERVED**, sau đó cho khách vào bàn để chuyển sang **OCCUPIED**. Với khách walk-in, lễ tân chọn trực tiếp một bàn trống và cho khách vào bàn mà không cần tạo reservation giả. Trạng thái bàn sau các thao tác này được đồng bộ sang màn hình nhân viên phục vụ.

3.5.4. Màn hình gọi món

Màn hình gọi món cho phép nhân viên phục vụ chọn món từ menu, thêm số lượng, nhập ghi chú và gửi order đến hệ thống. Một bàn có thể gọi thêm món nhiều lần trước khi thanh toán; các lần gọi mới được gắn vào order đang mở và tiếp tục phát sinh ticket cho bếp.

3.5.5. Màn hình bếp

Màn hình bếp hiển thị danh sách kitchen ticket đang chờ xử lý và danh sách các món đã hoàn thành/đã phục vụ. Nhân viên bếp có thể cập nhật trạng thái từng món hoặc từng ticket, từ đó hệ thống phản ánh lại trạng thái order cho khu vực phục vụ.

3.5.6. Màn hình lập bill và thanh toán

Màn hình bill hỗ trợ tổng hợp món đã gọi, tính subtotal, service fee, tip, discount và total. Trong phạm vi demo hiện tại, nhân viên phục vụ đảm nhiệm luôn thao tác thanh toán tại bàn, bao gồm thanh toán toàn bộ phần còn lại hoặc split bill theo số tiền từng lần thanh toán. Sau khi thanh toán đủ, trạng thái bill, order và bàn được cập nhật đồng bộ.

3.5.7. Luồng demo end-to-end

Với phạm vi hiện thực ưu tiên một *vertical slice* hoàn chỉnh, hệ thống tập trung vào luồng demo chính sau:

1. Lễ tân giữ bàn cho khách đặt trước hoặc cho khách walk-in vào bàn.
2. Nhân viên phục vụ chọn bàn đang có khách.
3. Tạo order và thêm món.
4. Hệ thống phát sinh kitchen ticket.
5. Nhân viên bếp cập nhật món đã sẵn sàng.

-
6. Nhân viên phục vụ đánh dấu món đã phục vụ và có thể thêm lần gọi món tiếp theo nếu khách gọi thêm.
 7. Nhân viên phục vụ tạo bill và xử lý thanh toán toàn bộ hoặc split bill.
 8. Bàn được giải phóng sau khi thanh toán hoàn tất.

Các màn hình minh họa có thể bao gồm:

- Màn hình đăng nhập
- Màn hình quản lý bàn
- Màn hình lễ tân cho đặt bàn, giữ bàn và khách walk-in
- Màn hình gọi món của nhân viên phục vụ
- Màn hình bếp hiển thị ticket
- Màn hình bếp hiển thị lịch sử món đã hoàn thành
- Màn hình lập bill và thanh toán
- Màn hình cập nhật trạng thái bàn sau thanh toán

3.6. Đóng gói và triển khai

Hệ thống có thể được đóng gói bằng Docker để đơn giản hóa quá trình triển khai và demo:

- Backend được đóng gói thành một Docker image.
- Database được chạy bằng Docker Compose.
- Flyway migration tạo schema và seed dữ liệu mẫu cho tài khoản, bàn, menu, order, bill và kitchen order.

Ví dụ cấu hình chạy:

- Backend: `localhost:3000`
- Frontend: `localhost:5173`
- Database: PostgreSQL container

4. Báo cáo phản ánh

4.1. Lợi ích của việc áp dụng SOLID

Việc áp dụng SOLID giúp nhóm xây dựng hệ thống với cấu trúc rõ ràng hơn, dễ bảo trì hơn và thuận lợi hơn khi chia việc giữa các thành viên. Nhờ SRP và DIP, các service và repository được tách biệt hợp lý, giúp giảm coupling và tăng khả năng kiểm thử. OCP đặc biệt hữu ích trong các phần như payment và discount, nơi nhu cầu mở rộng là rất rõ ràng.

4.2. Khó khăn gặp phải

Khó khăn lớn nhất của nhóm là xác định ranh giới module sao cho vừa đúng với domain, vừa không bị dàn trải. Ban đầu, một số trách nhiệm như quản lý bàn, order và billing có xu hướng bị gộp vào chung một service, làm tăng độ phức tạp. Luồng order sang bếp cũng cần xử lý cẩn thận để tránh mất kitchen ticket khi khách gọi món thêm nhiều lần; vì vậy hệ thống dùng event ở tầng ứng dụng và bổ sung ràng buộc/migration ở tầng dữ liệu như một lớp bảo vệ nhất quán. Ngoài ra, việc áp dụng DIP và ISP khiến thiết kế ban đầu mất nhiều thời gian hơn vì nhóm phải suy nghĩ kỹ về abstraction thay vì code trực tiếp.

4.3. Bài học rút ra

Qua quá trình thiết kế và hiện thực IRMS, nhóm nhận ra rằng SOLID không chỉ là các nguyên lý lý thuyết mà thực sự giúp tăng chất lượng thiết kế nếu được áp dụng đúng chỗ. Nhóm cũng rút ra bài học rằng với đồ án thời gian ngắn, việc chọn kiến trúc phù hợp với bối cảnh triển khai quan trọng hơn việc chạy theo một kiến trúc “hiện đại” nhưng vượt quá khả năng thực hiện.

5. Phân chia công việc

Bảng 2: Phân chia công việc giữa các thành viên

STT	Họ và tên	MSSV	Công việc chính
1	Nguyễn Nhật Tiến	2213059	Architecture lead và report: phân tích yêu cầu, thiết kế kiến trúc, UML, chuẩn interface, tổng hợp báo cáo
2	Lê Bảo Linh	2311849	Order Module: entity, service, repository, API order; phối hợp với Menu và Table
3	Trần Phú	1952924	Kitchen Module: kitchen order/ticket, status flow, kitchen API/UI; bám sự kiện từ Order Module
4	Phạm Đình Chương	2210400	Billing/Payment Module: bill, calculator, payment processor strategy; nhận dữ liệu từ Order Module
5	Trần Minh Đăng	2310735	Frontend và integration: màn hình demo cho server, kitchen, host và manager; tích hợp API của các module chính
6	Hà Đình Vinh	2313907	Database, testing và docs: schema, seed data, test cases, demo script và kiểm soát dữ liệu demo

6. Tổng kết

Dự án IRMS được thiết kế theo hướng **Layered Modular Monolith kết hợp Event-Driven nội bộ**, phù hợp với bài toán quản lý nhà hàng và bối cảnh thực hiện của nhóm. Kiến trúc này tạo ra sự cân bằng giữa tính học thuật trong thiết kế kiến trúc và tính khả thi trong hiện thực hệ thống.

Việc áp dụng các nguyên lý SOLID giúp hệ thống:

- giảm coupling giữa các module,
- nâng cao khả năng bảo trì,
- hỗ trợ mở rộng tính năng,
- tăng khả năng kiểm thử và tái sử dụng mã nguồn.

Trong quá trình thiết kế, thách thức lớn nhất là xác định ranh giới module hợp lý và tránh để một lớp hoặc một module ôm quá nhiều trách nhiệm. Tuy nhiên, chính quá trình đó cũng cho thấy giá trị thực tế của SOLID và tư duy kiến trúc mô-đun trong việc xây dựng một hệ thống phần mềm có cấu trúc rõ ràng, dễ mở rộng và phù hợp với môi trường phát triển nhóm.

7. Tài liệu tham khảo

1. Tài liệu assignment của học phần Software Architecture.
2. Martin, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
3. Fowler, M. *Patterns of Enterprise Application Architecture*.
4. Evans, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*.
5. Richardson, C. *Microservices Patterns*.